

QA & Testing Strategy Report

Project: project-zero (CPU-based Ternary LLM Inference Engine)
Language: C99 | LOC: ~3,714 (55 .c files, 30 .h headers)
Build: CMake 3.12+ / GNU Make | Platform: POSIX (Linux/macOS), Windows
Date: March 13, 2026 | Prepared by: Independent QA Architect

Executive Summary

This report presents a forensic-level, from-scratch Quality Assurance and Testing Strategy for the project-zero codebase: a high-performance, CPU-only LLM inference engine written in pure C99. The engine implements ternary weight quantization ($\{-1, 0, 1\}$), AVX2 SIMD acceleration, multi-threaded parallel matrix multiplication, a BPE tokenizer, sliding-window KV cache with int4/int8 compression, and chain-of-thought reasoning with thought filtering. All existing tests in the repository are deliberately ignored per the zero-trust mandate. This report constructs an independent, ground-up testing strategy covering 25+ distinct testing methodologies across functional, non-functional, and specialized domains.

Codebase Architecture Overview

Module	Key Files	Responsibility
core/	config.c, error.c, run_state.c, weights.c	Model config parsing, error handling, runtime state allocation, weight mapping from mmap
math/	ternary_matmul*.c, rmsnorm*.c, softmax*.c, rope.c, elementwise*.c, simd_dispatch.c, parallel_matmul.c	SIMD-dispatched math kernels: ternary matmul, RMSNorm, softmax, RoPE, elementwise ops, parallel distribution
memory/	aligned_alloc.c, mapped_file.c	64-byte SIMD-aligned allocation, cross-platform mmap with flock protection
threading/	thread_pool.c, cpu_probe.c	Epoch-based thread pool with mutex/cond synchronization, CPU topology probing
tokenizer/	tokenizer_load.c, tokenizer_encode.c, tokenizer_decode.c	BPE tokenizer: binary vocab loading, iterative merge encoding, byte-token decoding
transformer /	attention.c, ffn.c, forward.c, embedding.c, generate.c	Full transformer forward pass: GQA attention, SwiGLU FFN, autoregressive generation loop
sampling/	argmax.c, temperature.c, top_p.c, top_k.c, rng.c	Greedy/stochastic sampling: argmax, temperature scaling, nucleus (top-p), top-k, Xorshift64* RNG
kv_cache/	sliding_window.c, kv_compress.c, kv_strategy.c	Circular KV cache with pinned system prompts, int8/int4 absmax quantization, RAM-adaptive strategy

Module	Key Files	Responsibility
reasoning/	prompt_inject.c, thought_filter.c, reasoning_generate.c	Chain-of-thought: prompt augmentation, <think> tag state machine filter, reasoning-aware generation

Global Vulnerability & Risk Register

The following table identifies the highest-risk modules and functions across the entire codebase, ranked by potential impact. These areas demand the most rigorous testing focus across all methodologies.

Module / File	Risk Area	Severity	Rationale
memory/mapped_file.c	Memory-mapped I/O	CRITICAL	mmap with user-supplied paths. Improper cleanup leads to SIGBUS. flock race windows. Windows HANDLE-to-int cast loses data on 64-bit.
memory/aligned_allocator.c	Heap memory mgmt	CRITICAL	Custom allocator underpins every buffer. Overflow in tn_aligned_malloc (count*elem_size). Alignment validation edge cases.
core/weights.c	Untrusted file parsing	CRITICAL	weights_map() walks a binary blob with pointer arithmetic. Integer overflows in size calculations. ADVANCE macro bounds checking.
core/run_state.c	KV cache allocation	HIGH	Multi-factor size_t multiplication for massive buffers. tn_size_mul4 overflow paths. Double-free on partial alloc failure.
threading/thread_pool.c	Concurrency	HIGH	Epoch-based dispatch with mutex/cond. Race in task_remaining. Spurious wakeups. Deadlock if dispatch_epoch wraps to 0 (unsigned overflow).
tokenizer/tokenizer_encoder.c	Input processing	HIGH	BPE merge loop with malloc. Processes user-supplied text. O(n^2) merge scanning. prompt_len vs actual string length mismatch.
kv_cache/kv_compression.c	Quantization precision	HIGH	Lossy int8/int4 compression. Rounding errors accumulate over long sequences. Zero-vector edge case. Odd-dimension int4 packing.
reasoning/thought_filter.c	State machine	MEDIUM	Character-by-character tag parsing. Buffer overflow in output_buf via strlen accumulation. Nested/malformed <think> tags.
math/ternary_matmul_avx2.c	SIMD correctness	MEDIUM	AVX2 intrinsics with scalar tail. Weight values outside {-1,0,1} cause silent corruption. Alignment assumptions.
sampling/top_p.c	Heap allocation in hot path	MEDIUM	malloc(vocab_size * sizeof(ProbIndex)) per sample call. qsort on large arrays. Fallback to argmax on OOM hides failures.
transformer/generate.c	Generation loop	MEDIUM	Fixed 512-token prompt buffer. No bounds check on vocab_size index into logits. EOS detection hardcoded to token 2.
kv_cache/sliding_window.c	Circular buffer logic	MEDIUM	Modular arithmetic for position mapping. Degenerate case when circular_size<=0. write_head wrap-around correctness.

PART I: Core Execution Approaches

1. Black Box Testing

Independent Baseline:

Black box testing treats the engine as an opaque system. The tester has access only to the public API surface (function signatures in the 30 header files), the binary model file format specification, and the expected behavioral contract: given a model file and a text prompt, the engine produces a sequence of tokens representing coherent text. No knowledge of internal SIMD paths, memory layouts, or thread synchronization is assumed.

Strategy & Scope:

- Treat `generate()` and `generate_with_reasoning()` as the primary system-under-test entry points.
- Test with crafted binary model files of varying sizes (tiny: 4-dim, small: 64-dim, standard: 4096-dim).
- Validate output determinism: identical seed + temperature=0 must produce identical output across runs.
- Test boundary prompts: empty string, single character, maximum-length prompt (512 tokens), Unicode/multibyte input.
- Validate graceful failure: corrupted model files, truncated vocab files, zero-byte files, non-existent paths.

From-Scratch Test Cases:

```
// BB-001: Deterministic greedy decoding // Input: valid model + prompt "Hello" + temperature=0.0 //
Expected: identical output across 10 consecutive runs // BB-002: Empty prompt handling // Input: valid
model + prompt="" + max_tokens=50 // Expected: engine starts with BOS token (id=1), produces output //
BB-003: Corrupted model file // Input: random bytes file + any prompt // Expected: TN_ERR_INVALID_MAGIC
returned, no crash or SIGSEGV // BB-004: Maximum prompt length saturation // Input: prompt with 513+
tokens (exceeds prompt_tokens[512] buffer) // Expected: graceful truncation or error, no buffer
overflow
```

Vulnerability Focus: `generate.c` (fixed 512 prompt buffer), `config_read()` (magic/version validation), `mapped_file_open()` (file I/O error paths).

Tooling: Custom C test harness with process isolation (fork+exec), CUnit or Check framework for structured assertions, crafted binary fixtures via Python scripts.

2. White Box Testing

Independent Baseline:

White box testing leverages full knowledge of internal code structure, control flow, and data flow. Every branch, loop boundary, and error return path in the 3,714 lines of C code is a candidate for coverage. The primary focus areas are: (a) the 9 distinct error codes returned by `config_read()`, (b) the ADVANCE macro bounds-checking in `weights_map()`, (c) the 5-state finite automaton in `thought_filter_process()`, (d) the epoch-based thread pool synchronization, and (e) all SIMD scalar tail-loop correctness.

Strategy & Scope:

- Achieve 100% branch coverage on all validation functions: `config_read()`, `weights_map()`, `tokenizer_load()`.
- Trace every `TN_CHECK` and `TN_CHECK_ALLOC` macro to ensure each error return path is exercised.
- Cover all 5 states in the ThoughtFilter FSM with transition coverage: `PASSTHROUGH` to `BUFFERING_OPEN`, `BUFFERING_OPEN` to `THINKING`, `THINKING` to `BUFFERING_CLOSE`, `BUFFERING_CLOSE` to `OUTPUT`, and all mismatch fallback transitions.
- Test SIMD dispatch: verify `tn_simd_init()` sets all 8 function pointers, and that scalar/AVX2 paths produce numerically identical results within tolerance.
- Path coverage for `run_state_alloc()`: test each sequential allocation failure point (`x`, `xb`, `xb2`, `hb`, `hb2`, `q`, `att`, `logits`, `key_cache`, `value_cache`, `rope_freq`) to confirm proper cleanup via `run_state_free()`.

From-Scratch Test Cases:

```
// WB-001: Config validation branch coverage void test_config_dim_zero() { // Craft header with dim=0,
expect TN_ERR_INVALID_CONFIG uint8_t buf[36]; // magic(4)+version(4)+Config(28) write_u32(buf,
TN_MAGIC); write_u32(buf+4, TN_VERSION); Config cfg_in = {.dim=0, .hidden_dim=256, .n_layers=4,
.n_heads=4, .n_kv_heads=4, .vocab_size=100, .seq_len=64}; memcpy(buf+8, &cfg_in, sizeof(Config));
Config cfg_out; assert(config_read(&cfg_out, buf, sizeof(buf)) == TN_ERR_INVALID_CONFIG); } // WB-002:
Thought filter FSM transition coverage void test_filter_full_cycle() { ThoughtFilter f;
thought_filter_init(&f); char out[256]; // Feed "<think>internal</think>visible"
assert(!thought_filter_process(&f, "<think>", out)); assert(f.state == FILTER_THINKING);
assert(!thought_filter_process(&f, "internal", out)); assert(!thought_filter_process(&f, "</think>",
out)); assert(f.state == FILTER_OUTPUT); assert(thought_filter_process(&f, "visible", out));
assert(strcmp(out, "visible") == 0); }
```

Tooling: `gcov/lcov` for line and branch coverage reporting, GCC sanitizers (`-fsanitize=address,undefined`) already configured in `CMakeLists.txt` debug builds, Valgrind for heap analysis, custom fault-injection wrappers for `malloc/mmap`.

3. Manual Testing

Independent Baseline:

Manual testing targets workflows that require human judgment, intuition, and creative exploration that automated scripts cannot replicate. For this engine, manual testing is essential for: (a) evaluating the quality and coherence of generated text, (b) assessing the reasoning behavior of the `<think>` tag system, (c) verifying build system correctness across real hardware platforms, and (d) evaluating the usability of error messages and diagnostic output.

Workflows Requiring Human Execution:

- Text quality assessment: Run the engine on a reference model and evaluate output fluency, coherence, and factual consistency for diverse prompts (QA, creative writing, code generation).
- Reasoning trace review: Enable `generate_with_reasoning()`, capture stderr "Thinking..." output, verify the filter correctly hides chain-of-thought while preserving the final answer.
- Cross-platform build verification: Manually build on Linux (GCC, Clang), macOS (Apple Clang), Windows (MSVC, MinGW), and ARM (Raspberry Pi / Apple Silicon) to verify CMake architecture detection.
- Performance profiling review: Use `perf`/VTune to examine hotspots, verify SIMD utilization, and confirm thread pool is not over-subscribing CPU cores.
- Error message clarity: Trigger each of the 20 TernaryError codes and evaluate whether the returned string from `tn_error_str()` is actionable for an end-user.

Tooling: No frameworks required. Checklists in a spreadsheet or test management tool (TestRail, Zephyr). Screen recording for reasoning trace review sessions.

4. Automated Testing

Independent Baseline:

Automated testing must cover every deterministic, repeatable verification path and integrate into CI/CD pipelines. Given the C99 codebase with CMake, the automation strategy centers on: (a) unit test suites compiled and run via CTest, (b) sanitizer-enabled builds for every PR, (c) coverage gating at 90%+ line coverage, and (d) static analysis as a pre-merge gate.

CI/CD Pipeline Architecture:

Stage	Action	Gate Criteria
1. Lint	clang-format check, cppcheck --enable=all	Zero warnings
2. Build (Debug)	cmake -DCMAKE_BUILD_TYPE=Debug -DENABLE_TESTS=ON	Clean compilation with -Wall -Wextra -Wpedantic
3. Unit Tests	ctest --output-on-failure (with ASan+UBSan)	All tests pass, zero sanitizer errors
4. Coverage	gcov + lcov --rc lcov_branch_coverage=1	Line coverage >= 90%, Branch >= 80%
5. Static Analysis	clang-tidy, Coverity Scan	Zero critical/high defects
6. Build (Release)	cmake -DCMAKE_BUILD_TYPE=Release	Clean -O3 build, no NDEBUG-masked issues
7. Integration	End-to-end test with reference model	Deterministic output matches golden file
8. Fuzz	libFuzzer / AFL++ on config_read, tokenizer_load	No crashes in 10M iterations

Tooling: GitHub Actions / GitLab CI, CMake + CTest, gcov/lcov, clang-tidy, cppcheck, Coverity Scan, AFL++/libFuzzer, Valgrind (nightly).

PART II: Functional & Structural Testing

5. Unit Testing

Independent Baseline:

Every exported function across all 55 source files must have dedicated unit tests. The codebase contains approximately 75+ public functions spanning 9 modules. Unit tests must validate correct output for normal inputs, boundary conditions, and error paths in complete isolation from other modules.

Critical Unit Test Inventory:

Module	Function	Test Focus
math/	ternary_matmul()	Weights {-1,0,1} only, scale factor application, zero vector, single element, $n\%8 \neq 0$ tail
math/	rmsnorm()	Known reference values, zero vector (epsilon stability), large magnitude inputs
math/	softmax()	Uniform input (all equal), single large value dominance, numerical stability with large negatives
math/	apply_rope()	Position 0 (identity), known angle values, multi-head consistency, $n_heads \neq n_kv_heads$ (GQA)
math/	silu()	$x=0$ (expect 0), large positive (approaches x), large negative (approaches 0)
math/	vec_dot()	Orthogonal vectors (expect 0), parallel vectors, empty vector ($n=0$)
kv_cache/	kv_compress_to_8bit()	Round-trip fidelity (compress then decompress), zero vector, all-same values, alternating +/-127
kv_cache/	kv_compress_to_4bit()	Round-trip with expected quantization error < 15%, odd dimension handling, packed byte layout
kv_cache/	sw_map_position()	Positions within system prompt (identity), wrap-around after window_size, degenerate circular_size=0
kv_cache/	select_kv_strategy()	RAM threshold boundaries (1GB, 4GB, 16GB), seq_len < window defaults
sampling/	sample_argmax()	Single maximum, multiple equal maxima (first wins), vocab_size=1
sampling/	sample_top_p()	top_p=1.0 (full distribution), top_p=0.0 (greedy), OOM fallback path
sampling/	sample_top_k()	k=1 (greedy), $k > vocab_size$ (clamp), k=1024 (stack buffer boundary)
sampling/	rng_seed() + rng_float()	Seed=0 handling (must become 1), output in [0,1), deterministic replay from same seed
tokenizer/	tokenizer_encode()	Empty string, single char, BPE merge correctness, control token rejection (" $< $ ")

Module	Function	Test Focus
tokenizer/	tokenizer_decode()	Normal tokens, <0xHH> byte tokens, prev_token=1 space stripping, out-of-range token ID
core/	config_read()	All 9 validation checks: dim<=0, dim>65536, n_heads%n_kv_heads!=0, file too small, wrong magic, etc.
core/	weights_map()	Truncated data (ADVANCE fails at each layer), correct pointer offsets for 2-layer model
memory/	tn_aligned_alloc()	Alignment power-of-2 validation, size=0, alignment < sizeof(void*), huge allocation (OOM)
memory/	tn_size_mul_overflow()	SIZE_MAX * 2 (overflow), 0 * anything (no overflow), exact boundary SIZE_MAX/N * N
reasoning/	inject_reasoning_prompt()	Normal append, max_len exactly fits, max_len too small for suffix, max_len=0
reasoning/	thought_filter_process()	All 5 FSM states, partial tag matches, nested tags, empty token string
threading/	threadpool_create()	n=0 (returns NULL), n=1, n=large (e.g., 128), create then immediately destroy

From-Scratch Test Example:

```
// UT-MATH-001: ternary_matmul scalar correctness void test_ternary_matmul_identity() { // 4x4
identity-like ternary matrix: diagonal = 1, rest = 0 int8_t w[16] = {1,0,0,0, 0,1,0,0, 0,0,1,0,
0,0,0,1}; float x[4] = {1.0f, 2.0f, 3.0f, 4.0f}; float out[4]; ternary_matmul(out, x, w, 4, 4, 1.0f);
for (int i = 0; i < 4; i++) assert(fabsf(out[i] - x[i]) < 1e-6f); } // UT-MATH-002: ternary_matmul with
scale and negatives void test_ternary_matmul_neg() { int8_t w[4] = {1, -1, 0, 1}; // 1x4 row float x[4]
= {10.0f, 20.0f, 30.0f, 40.0f}; float out[1]; ternary_matmul(out, x, w, 4, 1, 2.0f); // Expected: 2.0 *
(10 - 20 + 0 + 40) = 2.0 * 30 = 60.0 assert(fabsf(out[0] - 60.0f) < 1e-6f); }
```

Tooling: Check (C unit testing framework), CMocka, or Unity. CTest integration via CMakeLists.txt. All tests compiled with ASan + UBSan.

6. Integration Testing

Independent Baseline:

Integration tests validate data flow and interaction between modules. The engine has clear integration seams: (a) config_read to weights_map (config drives weight pointer arithmetic), (b) tokenizer_encode to transformer_forward (token IDs flow into embedding lookup), (c) attention_forward to sliding_window (positions map through circular buffer), (d) SIMD dispatch to parallel_matmul (function pointers used in threaded context), and (e) the full generate pipeline (tokenizer to transformer to sampling to decode).

Key Integration Boundaries:

Integration Seam	Modules Involved	Risk / Test Focus
Config to Weights	config.c + weights.c + mapped_file.c	Config fields (dim, hidden_dim, kv_dim) must produce correct byte offsets in weights_map. Mismatch causes silent data corruption.
Tokenizer to Transformer	tokenizer_encode.c + forward.c + embedding.c	Token IDs from encoder must be valid indices into embedding table. Out-of-range ID causes OOB read.

Integration Seam	Modules Involved	Risk / Test Focus
Attention to KV Cache	attention.c + sliding_window.c + kv_compress.c	sw_map_position() output used as cache index. Incorrect mapping corrupts K/V entries for all subsequent tokens.
SIMD Dispatch to Thread Pool	simd_dispatch.c + parallel_matmul.c + thread_pool.c	Function pointers must be initialized before any threaded dispatch. NULL tn_ternary_matmul causes SIGSEGV.
Reasoning Pipeline	prompt_inject.c + thought_filter.c + generate.c	Injected prompt must encode correctly. Filter must hide content generated by the model.
RunState to All Modules	run_state.c + attention.c + ffn.c	Buffer sizes derived from config must match actual access patterns. xb2 reused as temp K buffer assumes dim >= kv_dim.

From-Scratch Test Case:

```
// IT-001: Full pipeline config-to-weights integration void test_config_weights_roundtrip() { // Create a minimal 2-layer model in memory Config cfg = {.dim=64, .hidden_dim=128, .n_layers=2, .n_heads=4, .n_kv_heads=2, .vocab_size=256, .seq_len=32}; // Calculate expected data size from config size_t expected_size = compute_weights_size(&cfg); uint8_t *data = calloc(1, expected_size); // Fill with known pattern, map weights TransformerWeights w; weights_alloc_pointers(&w, &cfg); TernaryError err = weights_map(&w, &cfg, (int8_t*)data, expected_size); assert(err == TN_OK); // Verify pointer offsets: w.wq[1] should be at expected offset assert(w.wq[1] == data + layer1_wq_offset); weights_free_pointers(&w); free(data); }
```

Tooling: Same as unit testing framework (Check/CMocka), plus custom fixture generators for synthetic model files. Memory-mapped test fixtures for reproducible integration tests.

7. System Testing

Independent Baseline:

System testing validates the complete, integrated application from model loading through text generation to output. This requires a real (or realistic synthetic) model file, a tokenizer vocabulary, and the full execution pipeline including SIMD initialization, thread pool creation, memory mapping, and autoregressive generation.

Strategy:

- Create a "tiny reference model" fixture: 4-dim, 2-layer, 32-vocab, 16-seq_len with known weights. This model is small enough to compute expected outputs by hand.
- Run the full generate() pipeline end-to-end and compare output against pre-computed golden reference.
- Test the complete lifecycle: mapped_file_open() to config_read() to weights_map() to run_state_alloc() to generate() to cleanup.
- Verify resource cleanup: after full execution, all mmap regions are unmapped, all aligned memory freed, thread pool destroyed with no leaked threads.
- System test with reasoning: generate_with_reasoning() with a model that produces <think> tokens, verify filter hides them.

Tooling: Python script to generate synthetic model binaries with known weights. Valgrind --leak-check=full on the complete run. Process-level timeout wrapper to catch infinite loops.

8. Functional Testing

Independent Baseline:

Functional testing verifies that the engine fulfills its business-level requirements: (a) it correctly loads and validates model files per the TNRY binary format, (b) it produces token sequences via autoregressive decoding with controllable randomness, (c) it supports multiple sampling strategies (greedy, top-p, top-k, temperature), (d) it handles long contexts via sliding window without OOM, and (e) reasoning mode correctly augments prompts and filters thinking tokens.

Functional Requirements Matrix:

Requirement	Acceptance Criteria	Test Approach
FR-01: Model Loading	Engine loads TNRY format files, rejects invalid magic/version	Supply valid/invalid model files, assert error codes
FR-02: Greedy Decoding	temperature=0 produces deterministic output	Run 10x with same seed, compare byte-for-byte
FR-03: Top-p Sampling	top_p in (0,1) restricts token distribution	Collect 1000 samples, verify distribution entropy decreases as top_p decreases
FR-04: Top-k Sampling	k limits candidate tokens	With k=1, output must match argmax; with k=vocab_size, full distribution
FR-05: Sliding Window	Long sequences beyond max_seq_len do not OOM	Generate 10x max_seq_len tokens, monitor RSS stays bounded
FR-06: KV Compression	int8/int4 compression reduces memory usage	Compare RSS with KV_FULL_F32 vs KV_QUANT_I8, assert 3.5x+ reduction
FR-07: Reasoning Mode	Prompt injection adds think instruction, filter hides thinking	Check augmented prompt contains suffix, verify content not in stdout
FR-08: Prompt Injection Defense	Input containing "< " is rejected	Pass "< endoftext >" as prompt, assert tokenizer_encode returns -1
FR-09: EOS Termination	Generation stops on EOS token (id=2)	Use model that always generates token 2, verify output length = 1
FR-10: Multi-threading	Thread pool accelerates matmul vs single-threaded	Time generate() with tp=NULL vs tp=threadpool_create(N), assert speedup > 1.5x for large dim

Tooling: CTest with labeled test suites (Functional), custom model fixtures, process memory monitoring via /proc/self/status (VmRSS).

9. Smoke Testing

Independent Baseline:

Smoke tests verify that the most critical paths are functional after every build. They should execute in under 30 seconds and catch catastrophic failures (segfaults, link errors, assertion failures) before more expensive test suites run.

Smoke Test Suite (8 tests, <30s total):

- SM-001: tn_simd_init() returns non-NULL string ("AVX2" or "Scalar").
- SM-002: tn_aligned_alloc(1024, 64) returns non-NULL, tn_aligned_free() does not crash.

- SM-003: `config_read()` with valid minimal header returns `TN_OK`.
- SM-004: `threadpool_create(2)` succeeds, `threadpool_destroy()` completes without hang.
- SM-005: `softmax()` on `{1,2,3}` produces valid probability distribution (sums to ~ 1.0).
- SM-006: `ternary_matmul()` on `1x1` matrix produces `scale * w * x`.
- SM-007: `rng_seed() + rng_float()` produces value in `[0,1)`.
- SM-008: `thought_filter_init() + process("hello")` returns "hello" in passthrough mode.

Tooling: CTest label "smoke", run as first CI stage. Failure blocks all downstream stages.

10. Sanity Testing

Independent Baseline:

Sanity tests are narrow, focused checks run after specific code changes to verify the modified functionality did not break. Unlike regression tests (which are broad), sanity tests target only the changed area.

Strategy:

- Maintain a mapping of source files to their sanity test suites. When `softmax_avx2.c` is changed, run only the softmax correctness tests (scalar vs AVX2 comparison).
- When `thread_pool.c` is changed, run the thread pool stress test (1000 rapid dispatches) and the deadlock detection test.
- When `kv_compress.c` is changed, run the int8/int4 round-trip fidelity tests.
- When `tokenizer_encode.c` is changed, run the BPE merge correctness and prompt injection rejection tests.
- Use `git diff --name-only` in CI to determine which sanity suites to trigger.

Tooling: CTest labels per module (e.g., `ctest -L math`, `ctest -L kv_cache`). CI script maps changed files to labels.

11. End-to-End (E2E) Testing

Independent Baseline:

E2E tests trace the complete data flow from user prompt input through all processing stages to final text output. For this engine, the E2E path is: text prompt (string) to `tokenizer_encode` (token IDs) to `transformer_forward` (logits, repeated) to sampling (next token) to `tokenizer_decode` (text output). E2E tests must verify the entire chain with no mocking.

E2E Test Cases:

```
// E2E-001: Full generation with tiny reference model // Setup: Load pre-built 4-dim model + tokenizer fixtures // Action: generate(cfg, w, s, tok, tp, "test", 10, 0.0, 1.0) // Verify: stdout output matches golden reference string // Verify: stderr shows "N tokens in Xs (Y tok/s)" // E2E-002: Reasoning mode end-to-end // Setup: Model trained to output "<think>step1</think>answer" // Action: generate_with_reasoning(cfg, w, s, tok, tp, "Q?", // 50, 0.0, 1.0) // Verify: stdout shows "answer" only, not "step1" // Verify: stderr shows "Thinking..." and think token count > 0 // E2E-003: Sliding window long context // Setup: Model with seq_len=64, max_seq_len=32 (sliding) // Action: generate(cfg, w, s, tok, tp, long_prompt, 100, 0, 1) // Verify: No crash, RSS stays under 2x initial allocation
```

Tooling: Python test orchestrator that builds fixtures, invokes the compiled binary, captures stdout/stderr, and compares against golden files. Pytest with subprocess.

12. Regression Testing

Independent Baseline:

Regression testing ensures that new changes do not break existing functionality. The strategy requires maintaining golden reference outputs that are version-controlled alongside the code.

Strategy:

- Numerical regression: Store expected output of `ternary_matmul`, `rmsnorm`, `softmax`, and `apply_rope` for fixed inputs in JSON golden files. Compare within epsilon (1e-5 for scalar, 1e-4 for SIMD).
- Output regression: Store the exact token sequence produced by greedy decoding of a reference model. Any change to `transformer_forward`, `attention`, or `FFN` must preserve this sequence.
- Performance regression: Track tok/s on a reference model in CI. Alert if throughput drops >10% compared to the previous baseline.
- Memory regression: Track peak RSS for reference model generation. Alert if memory usage increases >5%.

Tooling: Golden file comparison in CTest, benchmark tracking via Codspeed or custom CSV log, RSS monitoring via `/proc/self/status` parsing.

13. API Testing

Independent Baseline:

This engine exposes a C API (not HTTP). "API testing" validates the contract of every public function signature declared in the 30 header files. Each function is treated as an API endpoint with defined inputs, outputs, side effects, and error codes.

API Contract Verification Matrix:

API Function	Input Contract	Output Contract	Error Contract
<code>config_read(cfg, ptr, size)</code>	Non-NULL <code>cfg</code> , <code>ptr</code> ; <code>size</code> $\geq$ <code>header_size</code>	Populated Config struct	<code>TN_ERR_INVALID_MAGIC</code> , <code>CONFIG</code> , <code>VERSION</code>
<code>weights_map(w, cfg, data, size)</code>	Allocated <code>w</code> , valid <code>cfg</code> , <code>data</code> of correct size	Pointers into <code>data</code> region set	<code>TN_ERR_INVALID_WEIGHTS</code> on truncation
<code>run_state_alloc(s, cfg, max_seq)</code>	Zeroed <code>s</code> , valid <code>cfg</code> , <code>max_seq</code> $>$ 0	All buffers non-NULL, aligned to 64	<code>TN_ERR_OOM</code> on any alloc failure
<code>tokenizer_encode(t, text, len, tok, max)</code>	Loaded tokenizer, non-NULL <code>text</code>	Token count $\geq$ 0	-1 on NULL input, $<$ detected, overflow
<code>threadpool_create(n)</code>	$n >$ 0	Non-NULL ThreadPools*	NULL on $n \leq 0$, alloc failure, pthread failure
<code>threadpool_dispatch(tp, fn, arg, tot)</code>	Non-NULL <code>tp</code> and <code>fn</code> , <code>total</code> $>$ 0	Blocks until all work done	No-op on NULL <code>tp</code> , NULL <code>fn</code> , or <code>total</code> $\leq$ 0
<code>sample_top_p(logits, vs, p, rng)</code>	Non-NULL <code>logits</code> , <code>vs</code> $>$ 0, $0 < p \leq 1$	Valid token index in $[0, vs)$	Falls back to argmax on malloc failure
<code>mapped_file_open(mf, path)</code>	Non-NULL <code>mf</code> , valid path string	<code>mf->data</code> and <code>mf->size</code> set	<code>TN_ERR_FILE_OPEN</code> , <code>STAT</code> , <code>MMAP_FAILED</code>

Tooling: C unit test framework with assertion macros for each contract. Custom mock allocators to force OOM at specific call sites.

14. Database / Data Integrity Testing

Independent Baseline:

While the engine has no traditional database, it performs binary data I/O that is functionally equivalent: (a) model weight files are read-only databases of neural network parameters, (b) the KV cache is a write-heavy in-memory data store with compression and eviction, and (c) the tokenizer vocabulary is a persistent lookup table loaded from disk. Data integrity testing must verify that all data read, written, compressed, and decompressed maintains fidelity.

Data Integrity Test Cases:

- DI-001: Weight file integrity: Compute a checksum of weight data before and after `weights_map()`. Verify no writes occurred to the read-only mmap region.
- DI-002: KV cache int8 fidelity: Compress 10,000 random float vectors to int8 and decompress. Verify max error per element < 1% of absmax.
- DI-003: KV cache int4 fidelity: Same as DI-002 for int4. Verify max error < 15% of absmax (expected for 4-bit).
- DI-004: Tokenizer round-trip: For every token ID `0..vocab_size-1`, verify `tokenizer_decode(t, -1, id)` returns a valid, non-NULL string.
- DI-005: BPE encode-decode consistency: Encode a string, decode each token, concatenate. Verify the result equals the original string (accounting for BPE merge semantics).
- DI-006: KV cache circular overwrite: After the sliding window wraps, verify that pinned system prompt entries (positions `0..spl-1`) are never overwritten.

Tooling: Custom C test suite with statistical analysis (mean error, max error, P99 error) for quantization fidelity. Python reference implementation for expected quantization behavior.

PART III: Non-Functional Testing

15. Non-Functional Testing (General)

Independent Baseline:

Non-functional testing evaluates the system properties beyond correctness: performance, reliability, resource consumption, and operational behavior. For a C-based inference engine, the critical non-functional properties are: (a) memory safety (no leaks, no UB, no buffer overflows), (b) deterministic behavior (same inputs always produce same outputs for temperature=0), (c) graceful degradation under resource pressure, and (d) clean shutdown without resource leaks.

General Non-Functional Test Matrix:

Property	Metric	Target	Measurement Method
Memory Safety	ASan/UBSan violations	0 violations	Full test suite under ASan+UBSan
Memory Leaks	Valgrind definitely-lost bytes	0 bytes	Valgrind --leak-check=full on E2E test
Determinism	Bit-exact output reproducibility	100% for temp=0	10 runs with same seed, diff output
Resource Cleanup	Open file descriptors after run	Same as before run	Isof before/after, diff count
Crash Resilience	Segfaults on malformed input	0 crashes	Fuzz testing 10M iterations
Thread Safety	TSan data race reports	0 races	Full suite under ThreadSanitizer

Tooling: GCC/Clang sanitizers (ASan, UBSan, TSan, MSan), Valgrind (Memcheck, Helgrind, DRD), strace for syscall audit, Isof for FD leak detection.

16. Performance Testing

Independent Baseline:

Performance testing evaluates the engine throughput (tok/s), latency (time-to-first-token), memory footprint, and scalability across thread counts. The architecture provides multiple performance-critical paths: SIMD-accelerated matmul, parallel dispatch, and KV cache compression.

Performance Test Categories:

Load Testing:

- Measure tok/s with increasing model size: 64-dim (tiny), 512-dim (small), 2048-dim (medium), 4096-dim (standard).
- Measure tok/s with increasing thread count: 1, 2, 4, 8, 16 threads. Verify near-linear scaling up to physical core count.
- Measure tok/s with increasing sequence length: 32, 128, 512, 2048 tokens. Attention $O(n^2)$ should be visible.

Stress Testing:

- Generate maximum-length sequences (seq_len=1,048,576) with sliding window. Monitor RSS stays bounded.
- Rapid sequential generate() calls (1000x) without reinitializing state. Verify no memory growth (leak detection).
- Thread pool stress: dispatch 100,000 tiny tasks rapidly to expose epoch wraparound and race conditions.

Volume Testing:

- Load the largest feasible model (vocab_size=1,000,000) and verify mmap does not exhaust virtual address space.
- Process a 1M-character prompt through tokenizer_encode. Measure encoding time and memory usage ($O(n^2)$ BPE merge loop).

Benchmarking Targets:

Metric	Target (4096-dim, 32-layer)	Measurement
Throughput	> 5 tok/s single-threaded, > 15 tok/s 4-threaded	Wall-clock time / tokens generated
Time-to-first-token	< 500ms for 64-token prompt	clock_gettime before/after prompt processing
Peak RSS	< 2x model file size	/proc/self/status VmRSS
SIMD speedup	AVX2 matmul > 3x scalar	Benchmark both paths on same hardware
Thread scalability	> 0.7x linear scaling to 4 threads	tok/s(N) / (N * tok/s(1))

Tooling: Custom C benchmark harness with clock_gettime(CLOCK_MONOTONIC), perf stat for CPU cycle/IPC analysis, Valgrind Massif for heap profiling, Intel VTune for SIMD utilization.

17. Security Testing

Independent Baseline:

Security testing for a C-based inference engine must cover memory safety vulnerabilities, input validation weaknesses, and supply-chain risks. The engine processes untrusted binary files (model weights, tokenizer vocab) and untrusted text input (user prompts). All are potential attack vectors.

SAST (Static Application Security Testing):

Vulnerability Class	Affected Code	Detection Method
Buffer overflow	thought_filter output_buf (no size param), prompt_tokens[512] in generate.c	Coverity, CodeQL buffer-overflow queries
Integer overflow	tn_size_mul_overflow callers, weights_map ADVANCE macro arithmetic	clang-tidy bugprone-integer-overflow, manual review
Use-after-free	run_state_free() partial cleanup, weights_free then access	ASan, Coverity use-after-free checker

Vulnerability Class	Affected Code	Detection Method
Format string	printf("%s", piece) in generate.c (safe), but fprintf patterns	cppcheck formatString, gcc -Wformat
Null dereference	tn_ternary_matmul function pointer before tn_simd_init()	Coverity NULL_RETURNS, UBSan
Race condition	thread_pool.c task_remaining decrement, dispatch_epoch	TSan, Helgrind, manual mutex audit

DAST (Dynamic Application Security Testing):

- Fuzz config_read() with random binary blobs: target magic, version, and field validation paths.
- Fuzz tokenizer_load() with malformed vocab files: negative lengths, huge max_token_len, truncated entries.
- Fuzz tokenizer_encode() with adversarial text: null bytes, very long strings, all control characters.
- Fuzz weights_map() with truncated data: systematically reduce data_size and verify no OOB read.

SCA (Software Composition Analysis):

- The codebase has zero external dependencies beyond the C standard library, POSIX APIs, and Win32 APIs. This eliminates third-party supply chain risk.
- However, the build system uses CMake; verify CMake version for known CVEs. Verify the compiler (GCC/Clang) version for miscompilation bugs affecting security flags (-fstack-protector-strong, -D_FORTIFY_SOURCE=2).
- Audit the flock(LOCK_SH|LOCK_NB) in mapped_file_open for TOCTOU race conditions.

Architecture-Level Security Concerns:

- Prompt injection defense: tokenizer_encode() rejects inputs containing "<|" to prevent control token injection. Test with all known escape sequences.
- Model file as attack vector: A crafted model file with extreme dim/hidden_dim values could cause integer overflow in buffer allocation. Already partially mitigated by config_read validation bounds.
- Static decode buffer: tokenizer_decode.c uses a static byte_decode_buf[2] which is not thread-safe. If decode is called from multiple threads, this is a data race.

Tooling: Coverity Scan (free for open source), CodeQL (GitHub-native), AFL++ / libFuzzer for fuzz testing, Semgrep with C/C++ rules, -fstack-protector-strong -D_FORTIFY_SOURCE=2 (already in CMakeLists.txt).

18. Usability Testing

Independent Baseline:

As a C library/engine (not a GUI application), usability testing focuses on the developer experience: API ergonomics, error message clarity, documentation quality, build system usability, and diagnostic output.

Usability Evaluation Areas:

- API ergonomics: Evaluate whether the function signatures are intuitive. For example, does generate() have too many parameters? Should sampling config be encapsulated in a struct?

- Error messages: Each of the 20 TernaryError values must map to an actionable message. Test that `tn_error_str()` covers all codes and that the messages help diagnose the root cause.
- Build system: Test that `cmake -B build && cmake --build build` works on a fresh Linux system with only standard dev packages. Test the Makefile alternative path.
- Diagnostic output: Verify `config_print()` output is clear, performance stats (tok/s) are formatted correctly, and "Thinking..." indicator is visible on stderr.
- Header documentation: Verify all 30 header files have complete Doxygen-style comments for every public function, struct, and macro.

Tooling: Manual review checklist, developer survey (if team exists), Doxygen generation to validate comment coverage, CMake build on fresh Docker containers (Ubuntu, Alpine, Fedora).

19. Compatibility Testing

Independent Baseline:

The engine targets cross-platform compatibility (POSIX + Windows) with architecture-specific SIMD acceleration (x86 AVX2 + ARM NEON). Compatibility testing must verify correct builds and behavior across the entire platform matrix.

Compatibility Matrix:

Dimension	Variants to Test	Key Risk
OS	Ubuntu 22.04/24.04, Alpine (musl), macOS 13+, Windows 10/11	POSIX API differences, Windows stub paths
Compiler	GCC 11+, Clang 14+, Apple Clang, MSVC 2022	Warning differences, intrinsic support, C99 compliance
Architecture	x86-64 (AVX2), x86-64 (no AVX2), ARM64 (NEON), ARM32	SIMD dispatch correctness, -march=native portability
Build System	CMake 3.12 (minimum), CMake 3.28 (latest), GNU Make	GLOB CONFIGURE_DEPENDS, generator differences
libc	glibc 2.31+, musl 1.2+, MSVC CRT	posix_memalign availability, sysconf behavior
Memory	1GB RAM, 4GB RAM, 16GB+ RAM	KV strategy selection thresholds, OOM handling

Critical Compatibility Issues Found in Code Review:

- Windows HANDLE-to-int cast: In `mapped_file.c` line `mf->fd = (int)(intptr_t)hFile`: HANDLE is a pointer type. On 64-bit Windows, casting to int truncates the upper 32 bits. This is a latent bug.
- Windows thread pool stub: `thread_pool.c` Windows path returns NULL from `threadpool_create()` and no-ops all operations. Multi-threading is silently disabled on Windows.
- SIMD compile-time detection: `simd_dispatch.c` uses compile-time macros (`TN_HAS_AVX2`) instead of runtime CPUID. Binary built with `-mavx2` will crash on non-AVX2 hardware.

Tooling: GitHub Actions matrix build (OS x compiler x arch), Docker containers for musl/glibc, QEMU for ARM cross-compilation testing, Wine for Windows binary verification.

20. Accessibility Testing

Independent Baseline:

As a CLI/library with no GUI, WCAG compliance in the traditional sense does not apply. However, accessibility considerations for developer tools include:

- Screen reader compatibility: All output to stdout/stderr must be plain text, not relying on ANSI escape codes for semantic meaning.
- Color-independent diagnostics: The "Thinking..." indicator and performance stats must convey information via text, not color alone.
- Documentation accessibility: All .md files (README, IMPLEMENTATION_PLAN, etc.) should use proper heading hierarchy and alt text for any images.
- Error code documentation: Provide a lookup table mapping every TernaryError code to its meaning, accessible via the API itself (tn_error_str() provides this).

Status: The codebase uses no ANSI escape codes and outputs pure text. tn_error_str() provides text descriptions for all error codes. Accessibility risk is LOW.

21. Localization / Internationalization Testing

Independent Baseline:

The engine processes text through a BPE tokenizer that operates on raw bytes, making it inherently encoding-agnostic. However, internationalization testing must verify:

- UTF-8 multibyte handling: tokenizer_encode() processes individual bytes via prompt_len. Multibyte UTF-8 characters (Chinese, Arabic, emoji) will be split into single-byte tokens. Verify the BPE merge loop can reassemble them.
- Byte token decoding: The <0xHH> byte token mechanism in tokenizer_decode() supports arbitrary byte values including high-byte UTF-8 continuation bytes. Verify round-trip for all 256 byte values.
- Locale independence: The engine uses no locale-dependent functions (no setlocale, no toupper/tolower). Verify this remains true and that output is locale-invariant.
- Right-to-left text: BPE encoding is byte-order based (left-to-right byte stream). RTL text (Arabic, Hebrew) will encode correctly at the byte level but output order depends on the terminal. Verify no internal reordering occurs.
- Emoji and surrogate pairs: Test encoding/decoding of 4-byte UTF-8 sequences (emoji, mathematical symbols).

Tooling: Test fixtures with multilingual text samples (CJK, Arabic, Cyrillic, emoji). Python reference encoder for expected BPE behavior on multibyte input.

PART IV: Specialized & Exploratory Testing

22. Acceptance Testing (UAT)

Independent Baseline:

UAT criteria must be defined from the end-user perspective. For this engine, the end users are: (a) ML engineers deploying the engine for inference, (b) application developers integrating the C API, and (c) researchers evaluating ternary quantization quality.

Acceptance Criteria:

Stakeholder	Acceptance Criterion	Verification Method
ML Engineer	Engine loads any TNRY-format model and produces text	Load 3+ different model sizes, generate 100 tokens each
ML Engineer	Ternary quantization perplexity < 1.5x unquantized baseline	Benchmark on standard eval set (WikiText-2)
App Developer	API is documented, builds cleanly, links without issues	Fresh build on 3 platforms, Doxygen generation succeeds
App Developer	Error handling is complete; no unhandled crashes	1 hour of adversarial input testing with zero crashes
Researcher	SIMD acceleration provides measurable speedup	Benchmark scalar vs AVX2, demonstrate > 2x speedup on matmul
Researcher	Sliding window maintains coherent generation for long sequences	Generate 10,000 tokens, human review of output quality at end
All Users	Build-and-run requires only CMake + C compiler	Documented 3-command build process works on Ubuntu, macOS, Windows

Tooling: Acceptance test script (Python/bash) that automates the verification checklist. Human sign-off required for text quality and documentation assessments.

23. Exploratory Testing

Independent Baseline:

Exploratory testing applies unscripted, creative investigation to uncover edge cases and unexpected behaviors that formal test cases miss. The tester uses domain expertise and curiosity to probe areas where formal specifications are weak.

Exploration Charters:

- Charter 1 - Adversarial Prompts: Explore what happens when prompts contain model-specific control tokens, nested <think> tags, extremely long repeated characters, or binary data embedded in text.
- Charter 2 - Resource Exhaustion: Systematically reduce available memory (using ulimit -v) and observe how the engine degrades. Does it crash with SIGSEGV or return TN_ERR_OOM?
- Charter 3 - Concurrent Access: Launch multiple processes simultaneously reading the same mmap'd model file. Verify flock correctly prevents conflicts. What happens if one process truncates

the file?

- Charter 4 - Thread Pool Edge Cases: Create a thread pool with n=1000 (more threads than work items). Dispatch zero-item tasks. Dispatch while a previous dispatch is hypothetically still running (should be impossible but verify).
- Charter 5 - Quantization Extremes: Feed vectors of all-zero, all-max-float, alternating +/- FLT_MAX, NaN, and Inf to kv_compress functions. Check for division by zero, Inf scale factors.
- Charter 6 - Thought Filter Adversarial: Feed partial tags character-by-character: "<", "<t", "<th", "<thi", "<thin", "<think", "<think>". Verify the FSM handles each partial correctly and does not leak buffered characters.

Tooling: Session-based test management (SBTM). Tester notes captured in real-time. Time-boxed sessions of 60-90 minutes each.

24. Boundary Value Testing

Independent Baseline:

Boundary value analysis targets the limits of all input ranges, array bounds, and hardcoded constants. The codebase contains numerous boundaries that must be systematically probed.

Boundary Value Inventory:

Boundary	Min	Max	Code Location	Test Focus
Config.dim	1	65536	config.c:config_read	dim=0, dim=1, dim=65536, dim=65537
Config.vocab_size	1	1000000	config.c:config_read	vs=0, vs=1, vs=1000000, vs=1000001
Config.seq_len	1	1048576	config.c:config_read	seq=0, seq=1048576, seq=1048577
prompt_tokens[]	0	512	generate.c	n_prompt=0, 511, 512, 513 (overflow)
top_k stack buffer	1	1024	top_k.c	k=0, k=1, k=1024, k=1025 (OOB)
tag_buffer[]	0	15	thought_filter.c	Tag strings of length 15, 16 (overflow)
max_token_len	1	4096	tokenizer_load.c	len=0, len=4096, len=4097
thread count	1	INT_MAX	thread_pool.c	n=0, n=1, n=sysconf cores, n=10000
alignment	sizeof(void *)	arbitrary	aligned_alloc.c	align=0, 1, 7, 8, 64, 128
window_size	spl+1	INT_MAX	sliding_window.c	ws=spl (degenerate), ws=spl+1, ws=INT_MAX
temperature	0.0	INF	temperature.c	temp=0, 1e-9 (near-zero), 1.0, 100.0, INF
top_p	0.0	1.0	top_p.c	p=0.0, 0.001, 0.5, 0.999, 1.0

Critical Boundary Bugs Found in Code Review:

- top_k.c stack overflow: Uses int top_indices[1024] and float top_vals[1024] on stack. If k > 1024 (which the code does not prevent), this is a stack buffer overflow. The code only clamps k to vocab_size, not to 1024.

- `thought_filter output_buf`: The function appends characters via `strlen(output_buf) + index`. If the input token is long, `output_buf` (assumed 256 bytes by caller) can overflow. No size parameter is passed.
- `generate.c` prompt buffer: `int prompt_tokens[512]` is a fixed-size stack array. `tokenizer_encode` is called with `max_tokens=512`, but if the prompt is very long, encoding could still attempt to write beyond this.

Tooling: Parameterized test framework (CTest with fixtures), boundary value generators, stack canary verification via ASan.

25. Monkey / Random Testing

Independent Baseline:

Monkey testing bombards the engine with random, unexpected inputs to uncover unhandled exceptions, crashes, and undefined behavior. For a C program, this means random binary data for file inputs, random strings for text inputs, and random parameter values for function calls.

Strategy:

- Random model files: Generate files of random sizes (1 byte to 100MB) filled with random data. Feed to `mapped_file_open()` + `config_read()`. Zero crashes expected.
- Random prompts: Generate random byte sequences of length 0 to 10,000. Feed to `tokenizer_encode()`. Zero crashes expected.
- Random sampling parameters: Call `sample_top_p()` with random `top_p` in `[-100, 100]`, random `vocab_size` in `[0, 1000000]`, random logit values including NaN, Inf, -Inf.
- Random config values: Craft Config structs with random field values (including negative, zero, INT_MAX). Feed to `run_state_alloc()`. Verify either `TN_ERR_OOM` or `TN_ERR_INVALID_CONFIG`, never a crash.
- Random thread pool usage: Create/destroy pools with random sizes, dispatch random amounts of work, interleave dispatches with destroy calls.

Tooling: AFL++ in dumb mode (no instrumentation), custom C harness with `/dev/urandom` input, process isolation via fork to contain crashes. Run for 24+ hours.

26. Fuzz Testing

Independent Baseline:

Fuzz testing uses coverage-guided automated tools to generate inputs that maximize code path exploration. Unlike monkey testing (purely random), fuzz testing mutates inputs intelligently based on code coverage feedback. For C code, this is the most effective method for finding memory corruption vulnerabilities.

Fuzz Targets:

Target Function	Input	Risk	Priority
<code>config_read()</code>	Arbitrary binary blob (36+ bytes)	Integer overflow, invalid field combinations	P0 - Critical
<code>tokenizer_load()</code>	Arbitrary binary file	Heap overflow from malicious len fields, OOM	P0 - Critical

Target Function	Input	Risk	Priority
tokenizer_encode()	Arbitrary text + length	OOB read if prompt_len > strlen, merge loop	P0 - Critical
weights_map()	Binary blob + synthetic Config	Pointer arithmetic overflow, OOB read	P1 - High
thought_filter_process()	Arbitrary token strings in sequence	State machine corruption, output_buf overflow	P1 - High
kv_compress_to_8bit()	Arbitrary float vectors	Division by zero (amax=0), NaN propagation	P2 - Medium
kv_compress_to_4bit()	Arbitrary float vectors + odd dims	Packed byte boundary errors	P2 - Medium
inject_reasoning_prompt()	Arbitrary strings + max_len values	Truncation boundary, off-by-one	P2 - Medium

Fuzz Harness Example:

```
// fuzz_config_read.c - libFuzzer harness #include "core/config.h" #include <stdint.h> #include <stddef.h> int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) { Config cfg; // This should NEVER crash, regardless of input config_read(&cfg, data, size); return 0; } // Compile: clang -g -fsanitize=fuzzer,address // fuzz_config_read.c src/core/config.c src/core/error.c // -Iinclude -o fuzz_config // Run: ./fuzz_config -max_len=1024 -runs=10000000
```

Tooling: libFuzzer (Clang built-in), AFL++ (for GCC), Honggfuzz, OSS-Fuzz integration. All harnesses compiled with ASan + UBSan. Corpus seeded with valid model/vocab files.

27. Mutation Testing

Independent Baseline:

Mutation testing evaluates the quality of the new unit test suite by introducing small, deliberate source code changes (mutants) and verifying that at least one test fails for each mutation. A surviving mutant indicates a gap in the test suite.

High-Value Mutation Targets:

Mutation Category	Example Mutation	Target Code	Detection Expectation
Relational operator	Change > to >= in config_read dim check	config.c line: cfg->dim > 0	Config validation tests must catch dim=0 acceptance
Arithmetic operator	Change + to - in vec_add	elementwise.c: out[i] = a[i] + b[i]	vec_add unit tests must detect subtraction
Constant replacement	Change 127.0f to 126.0f in kv_compress	kv_compress.c: 127.0f / amax	Quantization fidelity tests must detect precision change
Return value	Return TN_OK instead of TN_ERR_INVALID_MAGIC	config.c: TN_CHECK(magic == TN_MAGIC)	Config error tests must detect missing validation
Boundary shift	Change i + 7 < n to i + 6 < n in AVX2 loops	elementwise_avx2.c	SIMD-vs-scalar comparison tests must detect off-by-one

Mutation Category	Example Mutation	Target Code	Detection Expectation
Negation	Remove negation in ternary_matmul: val += x[j] instead of val -= x[j]	ternary_matmul_scalar.c	Matmul tests with weight=-1 must catch sign error
Statement deletion	Remove sw_advance() call in attention.c	attention.c	Sliding window position tests must detect stale write_head
Conditional flip	Swap FILTER_THINKING to FILTER_OUTPUT in thought_filter	thought_filter.c	FSM transition tests must detect premature output

Target Mutation Score: $\geq 85\%$ killed mutants across all modules. Surviving mutants must be analyzed: either the test suite has a gap, or the mutant is semantically equivalent (benign).

Tooling: Mull (LLVM-based mutation testing for C/C++), Dextool mutate, or custom script-based mutation via sed transformations + rebuild + test. Integration with CMake test runner.

PART V: Consolidated Tooling & CI/CD Architecture

Master Tooling Recommendations

Category	Tool	Purpose
Unit Test Framework	Check, CMocka, or Unity	C-native test framework with assertion macros, test suites, fixtures
Build & Test Runner	CMake + CTest	Build system already in place; CTest integrates with test discovery
Coverage	gcov + lcov (+ genhtml)	Line, function, and branch coverage with HTML reports
Static Analysis	cppcheck, clang-tidy, Coverity Scan	Bug detection, code style, security vulnerability scanning
Dynamic Analysis	ASan, UBSan, TSan, MSan	Memory errors, undefined behavior, data races, uninitialized reads
Memory Profiling	Valgrind (Memcheck, Massif, Helgrind)	Leak detection, heap profiling, lock-order checking
Fuzz Testing	libFuzzer, AFL++, Honggfuzz	Coverage-guided input generation for crash discovery
Mutation Testing	Mull (LLVM), Dextool mutate	Test suite quality assessment via code mutation
Performance	perf, Intel VTune, custom C harness	CPU profiling, SIMD utilization, throughput benchmarking
CI/CD	GitHub Actions, GitLab CI	Automated build-test-analyze pipeline on every commit
Security (SAST)	CodeQL, Semgrep, Flawfinder	Pattern-based vulnerability detection in source code
Security (SCA)	N/A (zero dependencies)	No third-party libraries to audit
Documentation	Doxygen	API documentation generation from header comments
Fixtures	Custom Python scripts	Generate synthetic model files, tokenizer vocabs, test vectors

CI/CD Pipeline (Recommended)

Stage	Trigger	Actions	Gate
Pre-commit	git commit	clang-format check, cppcheck quick scan	Clean formatting + 0 errors
PR Build	Pull request	Debug build (ASan+UBSan) + full test suite + coverage	All pass, coverage >= 90%
PR Security	Pull request	clang-tidy + CodeQL + Semgrep	0 critical/high findings

Stage	Trigger	Actions	Gate
Merge	Merge to main	Release build + integration tests + benchmark	No perf regression > 10%
Nightly	Cron (daily)	Valgrind full suite + TSan + fuzz (1 hour) + mutation	0 leaks, 0 races, 0 fuzz crashes
Weekly	Cron (weekly)	Cross-platform matrix build (6 OS x 3 compilers) + fuzz (24hr)	All platforms build + pass

Conclusions & Recommendations

This report has constructed a complete, independent testing strategy for the project-zero codebase, covering 27 distinct testing methodologies across core execution approaches, functional/structural testing, non-functional testing, and specialized/exploratory testing. Key findings:

- 12 CRITICAL/HIGH risk areas identified in the Global Risk Register, with memory management, untrusted file parsing, and concurrency representing the highest risk.
- 3 latent bugs found during code review: (1) `top_k.c` stack buffer overflow when `k > 1024`, (2) `thought_filter output_buf` overflow with long tokens, (3) Windows `HANDLE` truncation in `mapped_file.c`.
- 75+ unit test targets identified spanning all 9 modules, with specific test cases provided for the most critical functions.
- 8 fuzz targets prioritized with libFuzzer harness code provided for the most critical entry point (`config_read`).
- Zero external dependencies eliminates SCA risk, but the build toolchain (CMake, compiler) must still be verified for known issues.
- Compile-time SIMD detection is identified as a portability risk; runtime `CPUID` probing is recommended.

The recommended implementation order is: (1) Build the unit test suite with the Check or CMocka framework, targeting 90%+ coverage. (2) Set up the CI/CD pipeline with sanitizer-enabled builds. (3) Implement the 8 fuzz harnesses and run continuously. (4) Address the 3 latent bugs found during this review. (5) Add integration and E2E tests with synthetic model fixtures. (6) Perform mutation testing to validate test suite quality. (7) Complete the cross-platform compatibility matrix.

This report was prepared as an independent, zero-trust analysis of the project-zero codebase. All existing tests were deliberately ignored. All recommendations are based solely on the raw source code and its architectural properties.

Repository: <https://github.com/shifulegend/project-zero>